

Lecture :- Arrays 2

Agenda

find k in row and col wise
sorted matrix

Boundary print of matrix

Spiral order of a matrix

Sum of all submatrices sum.

Ques Given a row and col wise sorted matrix.

find out whether k is present or not. (☆☆☆)

	0	1	2	3
0	-5	-2	1	13
1	-4	0	3	14
2	-3	2	6	18

k=13 true

k=2 true

k=15 false

Brute force:

```
boolean search(int[][] mat, int k) {  
    n = mat.length;  
    m = mat[0].length;  
    for(i=0; i<n; i++) {  
        for(j=0; j<m; j++) {  
            if(mat[i][j] == k) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

TC: $O(n*m)$

SC: $O(1)$

Approach 2

	0	1	2	3
0	5	10 ←	15 ←	20
1	6	12 ↓	18	24
2	7 ←	14 ↓	21	28
3	8 ↓	16	24	32

$k=13$
false

	0	1	2	3
0	5	10	15 ←	20
1	6	12 ←	18 ↓	24
2	7	14 ↓	21	28
3	8	16 ↓	24	32

$k=16$
true

	0	1	2	3
0	5	10	15	20
1	6	12	18	24
2	7	14	21	28
3	8	16	24	32

$k=32$
true

Algorithmic code

```
boolean search(int[][] mat, int k) {  
    n = mat.length;  
    m = mat[0].length;  
    i = 0;  
    j = m-1;  
    while(i < n && j >= 0) {  
        if(mat[i][j] == k) {  
            return true;  
        }  
        if(mat[i][j] > k) {  
            j--;  
        } else {  
            i++;  
        }  
    }  
    return false;  
}
```

TC: $O(n+m)$ linear
SC: $O(1)$

Problem 3

Boundary level printing

Given $mat[n][n]$, print boundary elements in clockwise direction.

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

output

1	2	3	4	5
10	15	20	25	
24	23	22	21	
16	11	6		

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

output

1	2	3
6	9	
8	7	
4		

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Approach

mat[5][5]

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

$i=0$ & $j=0$

Yellow

i	j
0	0
0	1
0	2
0	3
0	4

stop

$j++$

$i=0$ & $j=4$

Red

i	j
0	4
1	4
2	4
3	4
4	4

stop

$i++$

$i=4$ & $j=4$

Green

i	j
4	4
4	3
4	2
4	1
4	0

stop

$j--$

$i=4$ & $j=0$

Silver

i	j
4	0
3	0
2	0
1	0
0	0

stop

$i--$

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Algorithmic code

```
void boundaryPrint(int[][] mat) {  
    n = mat.length;  
    i = 0, j = 0
```

Oth row

```
for(k=0; k<n-1; k++) {  
    print(mat[i][j]);  
    j++;  
}
```

Last col.

```
for(k=0; k<n-1; k++) {  
    print(mat[i][j]);  
    i++;  
}
```

Last Row

```
for(k=0; k<n-1; k++) {  
    print(mat[i][j]);  
    j--;  
}
```

Oth col

```
for(k=0; k<n-1; k++) {  
    print(mat[i][j]);  
    i--;  
}
```

```
}
```

TC: $O(n) + O(n) + O(n) + O(n) = O(4n) = O(n)$

SC: $O(1)$

Spiral order matrix

Medium

Given $mat[n][n]$, print spiral order of matrix in clockwise direction.

	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36

output

```

1  2  3  4  5  6
12 18 24 30 36
35 34 33 32 31
25 19 13 7
8  9  10 11
17 23 29
28 27 26
20 14
15 16
22 21
    
```

```

13  14  12  8
9   1   2   7
0   4   3   0
10  5   6  11
    
```

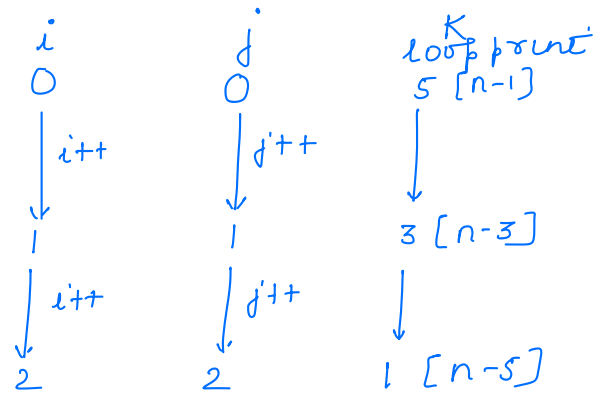
```

13  14  12
8   7   0
11  6   5
10  0   9
1   2
3   4
    
```

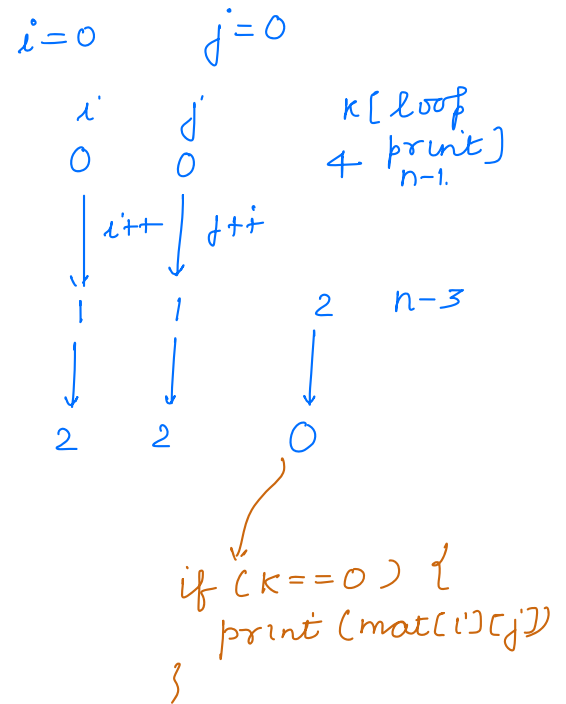

Approach

mat[6][6]

	0	1	2	3	4	5
0	1	2	3	4	5	6
1	7	8	9	10	11	12
2	13	14	15	16	17	18
3	19	20	21	22	23	24
4	25	26	27	28	29	30
5	31	32	33	34	35	36



	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25



Algorithmic code

```
void spiralOrder(int[][] mat) {  
    n = mat.length;  
    i = 0    j = 0    loopPrint = n-1  
    while (loopPrint >= 0) {  
        if (loopPrint == 0) {  
            print(mat[i][j]);  
            break;  
        }  
  
        0th row      loopPrint  
        for(k=0; k<n-1; k++) {  
            print(mat[i][j]);  
            j++;  
        }  
  
        Last col.   loopPrint  
        for(k=0; k<n-1; k++) {  
            print(mat[i][j]);  
            i++;  
        }  
  
        Last Row  
        for(k=0; k<n-1; k++) {  
            print(mat[i][j]);  
            j--;  
        }  
  
        0th col     loopPrint  
        for(k=0; k<n-1; k++) {  
            print(mat[i][j]);  
            i--;  
        }  
  
        i++;  
        j++;  
        loopPrint -= 2;  
    }  
}
```

TC: $O(n^2)$
SC: $O(1)$

Submatrices and its identification

Subarray is continuous and ordered part of array

Submatrix is " " " " " matrix.

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20
4	21	22	23	24	25

Submatrices

1	2	3	4
---	---	---	---

Subarrays:

Break: 8:16 - 8:26

Problem 5

Sum of all submatrices sum

Given $mat[n][m]$, determine sum of all the possible submatrices.

	0	1	2
0	4	9	6
1	5	-1	2

All submatrices

$$[4] = 4$$

$$[9] = 9$$

$$[6] = 6$$

$$[5] = 5$$

$$[-1] = -1$$

$$[2] = 2$$

$$[4 \ 9] = 4 + 9$$

$$[4 \ 9 \ 6] = 4 + 9 + 6$$

$$\begin{bmatrix} 4 & 9 \\ 5 & -1 \end{bmatrix} = 4 + 9 + 5 + (-1)$$

$$\begin{bmatrix} 4 & 9 & 6 \\ 5 & -1 & 2 \end{bmatrix} = 4 + 9 + 6 + 5 + (-1) + 2$$

$$[9 \ 6] = 9 + 6$$

$$\begin{bmatrix} 9 & 6 \\ -1 & 2 \end{bmatrix} = 9 + 6 + (-1) + 2$$

$$\begin{bmatrix} 4 \\ 5 \end{bmatrix} = 4 + 5$$

$$\begin{bmatrix} 9 \\ -1 \end{bmatrix} = 9 + (-1)$$

$$\begin{bmatrix} 6 \\ 2 \end{bmatrix} = 6 + 2$$

$$[-1 \ 2] = -1 + 2$$

$$[5 \ -1] = 5 + (-1)$$

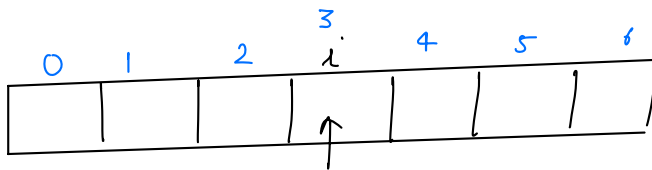
$$[5 \ -1 \ 2] = 5 + (-1) + 2$$

$$Ans = 4 * 6 + 9 * 8 + 6 * 6 + 5 * 6 + (-1) * 8 + 2 * 6$$

Approach

$$\text{Ans} = 4 * 6 + 9 * 8 + 6 * 6 + 5 * 6 + -1 * 8 + 2 * 6$$

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 $M[0][0]$ $M[0][1]$ $M[0][2]$ $M[1][0]$ $M[1][1]$ $M[1][2]$



i is part of how many subarrays?
 start indices i can be part of = 0, 1, 2, 3
 end " " " " " " = 3, 4, 5, 6
 $\text{ans} = (i+1) * (n-i)$

Challenge

	0	1	2	3	4	5	6	7	8	9
0	✓	✓	✓	✓						
1	✓	✓	✓	✓						
2	✓	✓	✓	✓						
3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
4				✓	✓	✓	✓	✓	✓	✓
5				✓	✓	✓	✓	✓	✓	✓
6				✓	✓	✓	✓	✓	✓	✓
7				✓	✓	✓	✓	✓	✓	✓

No of start indices where (3,3) can be
 part of $\Rightarrow (i+1) * (j+1)$
 $\downarrow$
 top left

No of end indices where (3,3) can be
 part of $\Rightarrow (n-i) * (m-j)$
 $\downarrow$
 bottom right

```

int totalSubMatricesSum(int[][] mat) {
    total = 0;
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            topLeft = (i+1) * (j+1)
            bottomRight = (n-i) * (m-j);
            ocl = topLeft * bottomRight
            total += mat[i][j] * ocl;
        }
    }
    return total;
}

```

TC: $O(n^2) \simeq O(n*m)$
 SC: $O(1)$

Thankyou 😊